



Assignment 2

Problem Definition

In this assignment, you will implement the **Word2Vec** model using **SkipGram with Negative Sampling** to build learned embeddings. You will also use them for building a **Neural Network** and **Hidden Markov Model** for **Named Entity Recognition Task**.

Part 1

You are required to implement a **Word2Vec model** using the **Skip-Gram with Negative Sampling (SGNS)** approach. Follow the steps below carefully:

1. Dataset

- Use the [lhoestq/conll2003](#) dataset from **Hugging Face**.
- Load the dataset and extract all text data (train, validation, and test sets).
- Use the **entire dataset** for training your model.

2. Preprocessing

- Apply the needed preprocessing.

3. Model Implementation

- Implement the **Skip-Gram** architecture where the model predicts context words given a target word.
- Use **Negative Sampling** to efficiently approximate the softmax output and speed up training.

4. Training

- Train your model on the full corpus until convergence.
- Monitor loss to ensure stable learning.

5. Saving Embeddings

- After training, **save the learned word embeddings**.



- These embeddings will be used in the **next part of the assignment**.

6. Word Analogy Function

- Implement a **function for word analogy**, which should take three input words (e.g., A, B, A^*) and predict the fourth word (B^*).

Part 2

In this part of the assignment, you will use the **word embeddings learned in Part 1** to build and evaluate **Named Entity Recognition (NER)** models using **two different approaches**:

1. A **Neural Network–based model**
2. A **Hidden Markov Model (HMM)–based model**

NER Using Neural Network

- Implement a **Feed-Forward Neural Network model**
- Use the **three splits** of the data:
 - **Train set** → for training the model
 - **Validation set** → for tuning hyperparameters
 - **Test set** → for final evaluation
- The input to the model should be the **word embeddings** from Part 1.
- The output should be the **predicted entity tags** for each token.
- Evaluate the model using metrics such as **accuracy, precision, recall, and F1-score**.

NER Using Hidden Markov Model (HMM)

- Implement a **Hidden Markov Model–based NER system**.
- Use only the **train split** for training and the **test split** for evaluation.
- Compute transition and emission probabilities from the training data.
- Use the **Viterbi algorithm** for decoding the most likely sequence of entity tags for the test sentences.
- Evaluate the performance using metrics such as **accuracy, precision, recall, and F1-score**.
- Compare results to the **Feed-Forward Neural Network model**.



Deliverables

- This project must be done in teams of three or four.
- Teams of two are **NOT** allowed.
- You should deliver a notebook containing all your code, plots, results, comparisons, and any materials you used.